

FOUNDATIONS COURSE • MODULE 01

DATA ENGINEERING: FROM RAW EVENTS TO RELIABLE DECISIONS

Building dependable, automated data
systems for production scale

CORE ARCHITECTURE

PRODUCTION READY

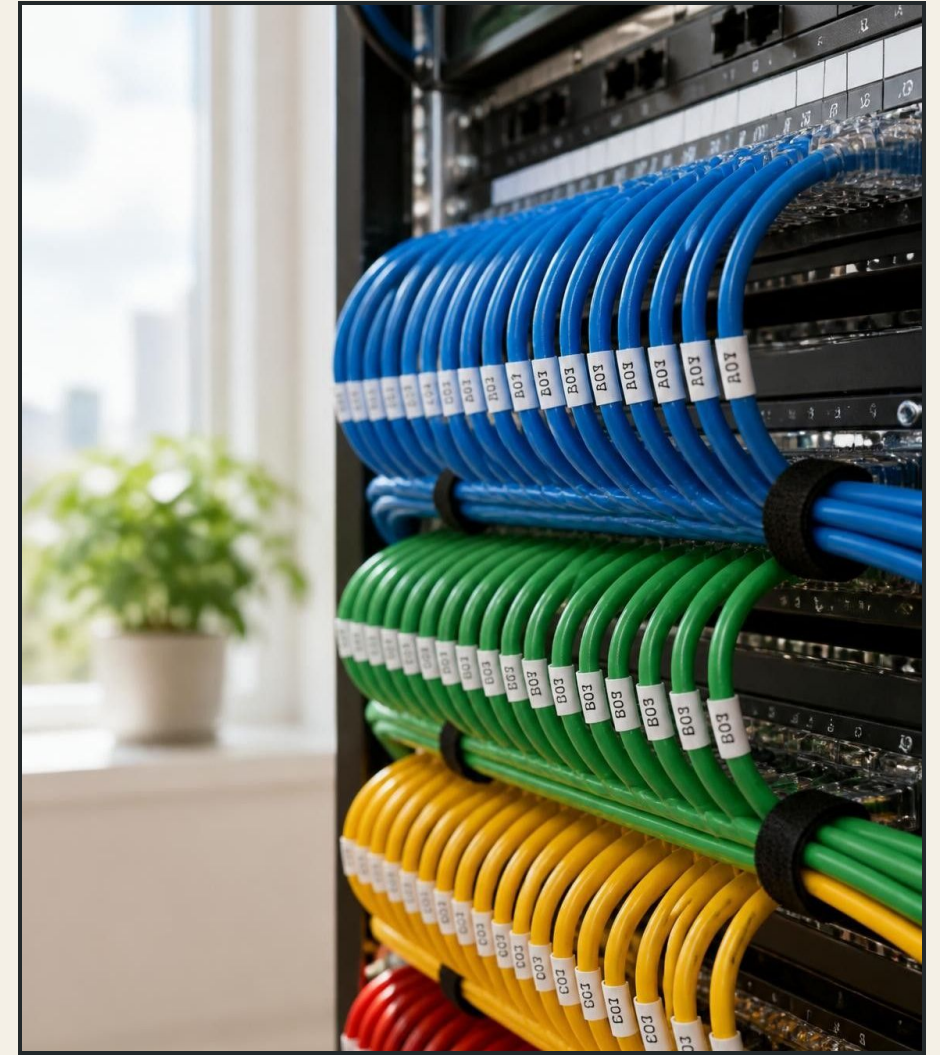
A practical engineering guide to pipelines, storage models,
orchestration, and operational trust.

BhavPath-Bhavya Ponduri

WHAT DATA ENGINEERS ACTUALLY BUILD

Automated systems move raw data from source systems into usable formats for analysts, data scientists, models, and dashboards.

The job balances freshness, reliability, cost, security, and trust.

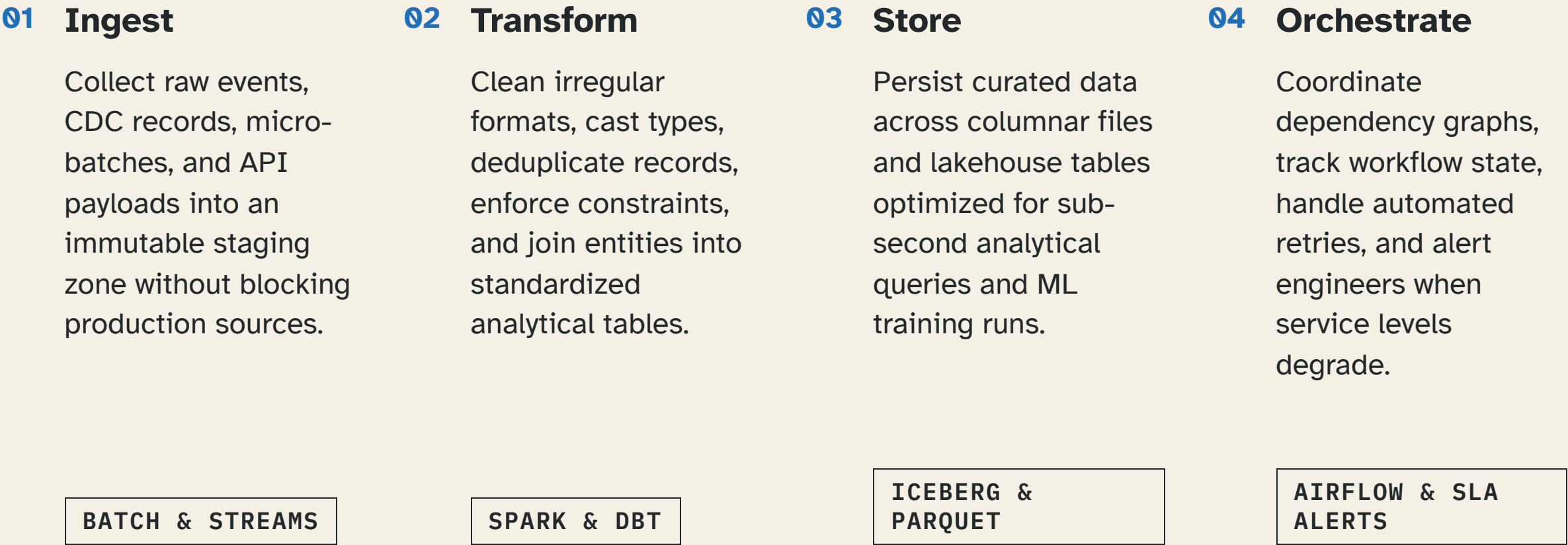


BhavPath-Bhavya Ponduri

Made with GAMMA

THE PIPELINE ANATOMY IS SIMPLE ON PURPOSE

Every data platform breaks down into four essential stages that turn erratic raw inputs into reliable decision engines.



Keep boundaries clean: decoupling each stage isolates failures and allows independent scaling.

BhavPath-Bhavya
Ponduri

BATCH OR STREAMING: CHOOSE FOR THE DECISION, NOT THE HYPE

Batch

Move data in intervals when minutes or hours of latency are acceptable.

Streaming

Process events continuously when freshness changes the action.

Hybrid

Combine both when operational and analytical needs differ.

Rule of thumb: default to batch unless the business takes immediate automated action on fresh data.

BhavPath-Bhavya
Ponduri

ETL AND ELT PLACE THE WORK IN DIFFERENT LAYERS

ETL

Transform before loading

Useful when quality checks must happen early or target compute is limited.

ELT

Load first; transform in the warehouse or lakehouse

Useful when compute is flexible and models evolve.

The right choice follows platform maturity, cost, and recovery needs.

BhavPath-Bhavya Ponduri

STORAGE LAYERS TURN RAW DATA INTO TRUSTED PRODUCTS

The medallion architecture progressively refines unstructured dumps into verified, business-ready models.

BRONZE • RAW

Source Fidelity

Append-only landing zone. Captures complete source payloads, ingestion timestamps, and raw JSON. Enables historical replay and forensic recovery when upstream formats drift.

SILVER • CLEANED

Standardized Entities

Deduplicated, schema-enforced, and validated. Unpacks nested structures, standardizes timestamps, resolves entity identities, and strips invalid characters.

GOLD • SERVING

Curated Products

Business-level aggregations, dimensional star schemas, and feature stores. Modeled directly for high-speed dashboard queries, executive metrics, and ML inference.

Each tier serves a distinct audience: Bronze for engineers, Silver for analysts, Gold for products and decisions.

BhavPath-Bhavya
Ponduri

Made with GAMMA

DISTRIBUTED PROCESSING MAKES SCALE MANAGEABLE

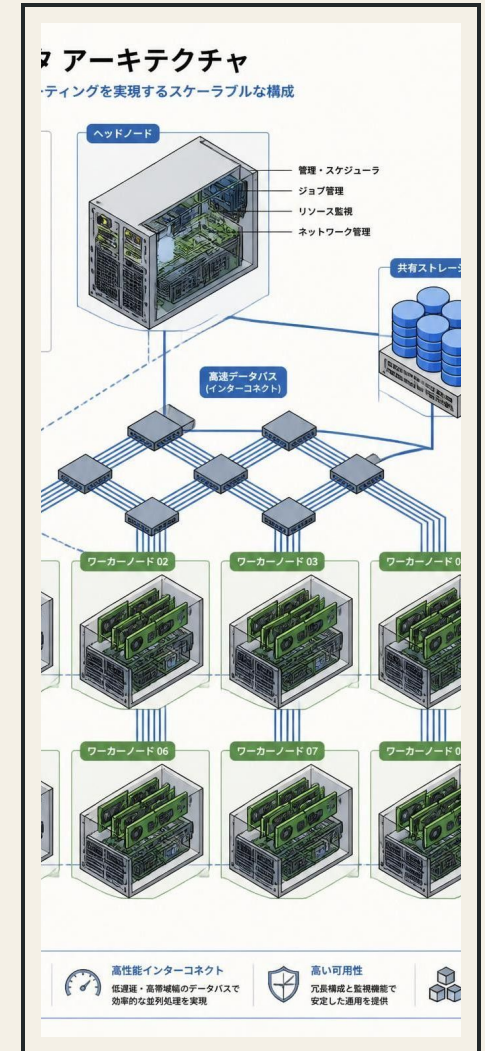
Partition & parallelize

Split large workloads into parallel partitions, not one machine. When data volume or transformation complexity outgrows a single process, engines such as Apache Spark distribute transforms across dozens of executor nodes.

Plan for shuffle & skew

Wide exchanges (joins, distinct groupings) move terabytes across network switches; optimize keys to minimize shuffle latency. Uneven key distribution concentrates work on one sluggish node; salting and broadcast joins balance cluster utilization.

Design for partitioning, skew, retries, and cost-aware compute. Use distributed clusters only when single-node tooling (DuckDB, Polars) cannot handle the data size.



ORCHESTRATION IS THE OPERATING SYSTEM OF THE PIPELINE

Managing Directed Acyclic Graphs (DAGs) to coordinate tasks, handle retries, and guarantee delivery SLAs.



Dependency Graphs

DAGs ensure downstream aggregations never execute until upstream ingestion passes validation criteria.

APACHE AIRFLOW • DAGSTER



Automated Backfills

Parameterizing execution intervals enables deterministic re-runs of historical windows after schema updates or logic fixes.

DBT • DECLARATIVE MODELS



Live Observability

Surfaces task run durations, upstream delay, data freshness drift, and failure traces with automated alerts to team on-call.

DATADOG • OPENLINEAGE

Cron triggers scripts; true orchestrators manage dependencies, retry exponential backoff, and state recovery.

BhavPath-Bhavya
Ponduri

Made with GAMMA

TRUST COMES FROM QUALITY, GOVERNANCE, AND LINEAGE

DIMENSION	CHECKS, CONSEQUENCE, AND TOOLING
Completeness	Row count checks & null thresholds; under-reporting revenue or dropped event traffic. Great Expectations.
Uniqueness	Primary key & composite duplicate tests; double-counted billing and skewed conversion metrics. dbt test.
Freshness	Maximum ingestion delay tolerances; stale dashboard data can mislead operational decisions. Monte Carlo / Soda.
Lineage	Column-level dependency mapping; untracked breaking schema changes downstream. OpenLineage.

Also test validity and referential integrity; track dataset owners, origins, and who can use each dataset. In production, use version control, CI/CD, access controls, and observability.

Test data at the boundary: catch bad records before Gold tables to protect team credibility.

BhavPath-Bhavya
Ponduri

A PRODUCTION-READY DATA ENGINEER DESIGNS FOR FAILURE

Systems will break, networks will drop packets, and source schemas will drift. Build defensively.



Idempotency

Design pipeline writes using partition overwrites or upserts so retrying a failed task never produces duplicate records.

MERGE & OVERWRITE



Data Contracts

Enforce schema contracts at the API and CDC layer to block silent upstream breaking changes before ingestion runs.

PROTOBUF & JSON
SCHEMA



Checkpointing

Preserve source offsets and state checkpoints to allow deterministic recovery and backfilling without restarting from scratch.

STATE STORE & REPLAY



Service Levels

Measure performance with explicit Data SLAs: track freshness lag, error budgets, and cloud compute costs alongside uptime.

SLAS & COST BUDGETS

Senior data engineering mindset: assume every component will fail, and make recovery completely routine.

BhavPath-Bhavya
Ponduri

HOW TO USE THIS LEARNING PATH

Move from concepts to production practice: understand the data lifecycle, choose ingestion and storage patterns, build transformations, then operate the system with quality, security, and recovery in mind.

Learn the why

Start with decisions, data contracts, and failure modes before choosing tools.

Practice the how

Translate each concept into a small pipeline, a test, and an operational check.

FOUNDATIONS: DATA IS A PRODUCT OF A SYSTEM

Data engineering connects producers, storage, computation, consumers, and operational controls. A useful dataset is not only a collection of rows; it has a purpose, an owner, a definition, and an expected service level.

- **Producers** create events, records, files, or API responses.
- **Platforms** move, store, transform, and serve those assets.
- **Consumers** make decisions through reports, models, products, or automation.

Design from the decision backward: what must be true for the consumer to trust the output?

BhavPath-Bhavya Ponduri

DATA SOURCES: UNDERSTAND THE SHAPE AT THE EDGE

Operational databases

Tables, transactions, constraints, and indexes. Reads must respect production workload and consistency rules.

Applications and services

Events, logs, traces, webhooks, and API responses. Definitions can change with deploys.

For every source, record ownership, extraction method, update behavior, sensitivity, and the business meaning of key fields.

Files and partner feeds

CSV, JSON, XML, spreadsheets, object storage drops, and managed transfers. Schema drift is common.

Human-entered systems

CRM, support, and finance tools often contain corrections, backfills, and ambiguous status values.

BhavPath-Bhavya Ponduri

DATA CONTRACTS MAKE PRODUCER CHANGES VISIBLE

A data contract states what a producer promises and what consumers may rely on. It should cover field names, types, requiredness, allowed values, event meaning, time semantics, and compatibility expectations.

CONTRACT PART	QUESTION TO ANSWER
Schema	Which fields exist, with what types and null rules?
Semantics	What does an event mean, and when is it emitted?
Compatibility	Can a consumer survive an additive or breaking change?

BhavPath-Bhavya Ponduri

INGESTION PATTERNS: CHOOSE THE BOUNDARY DELIBERATELY

Pull

Scheduled jobs query a source or call an API.
Control is centralized, but rate limits and incremental keys matter.

Push

Producers send events or files to an endpoint or queue. Latency improves, while retries and idempotency become essential.

Good ingestion preserves source evidence, captures extraction metadata, isolates malformed records, and makes replay possible.

BhavPath-Bhavya Ponduri

INCREMENTAL LOADS AND CHANGE DATA CAPTURE

Full extracts are simple but expensive as data grows. Incremental patterns move only what changed, using a reliable cursor such as an update timestamp, monotonically increasing key, log position, or source-native change feed.

- Store the last successful cursor separately from the data.
- Handle late updates, deletes, clock skew, and replays.
- Make the merge operation idempotent so a retry does not duplicate facts.
- Keep a backfill path for correcting historical logic.

Never assume an update timestamp alone proves that no earlier record can arrive late.

BhavPath-Bhavya Ponduri

BATCH PROCESSING: OPTIMIZE FOR COMPLETENESS AND REPEATABILITY

Batch jobs collect a bounded set of records and process them together. They are a strong fit for scheduled reporting, periodic reconciliation, historical backfills, and workloads where a small delay is acceptable.

Batch design questions

What is the unit of work? What closes the window? Which inputs are complete?

Operational safeguards

Use checkpoints, manifests, deterministic outputs, retries, and explicit rerun behavior.

BhavPath-Bhavya Ponduri

STREAMING: OPTIMIZE FOR CONTINUOUS DECISIONS

Streaming systems process records as they arrive, often through a durable log or broker. The hard parts are not only low latency; they are ordering, duplicates, late events, state, and recovery.

- Define event time separately from processing time.
- Use keys to control partitioning and preserve order where needed.
- Choose delivery and processing guarantees intentionally.
- Design windows and watermarks for late data.

Streaming is justified by a decision that benefits from fresher data, not by novelty alone.

BhavPath-Bhavya Ponduri

ETL VERSUS ELT: WHERE SHOULD TRANSFORMATION HAPPEN?

DIMENSION	ETL	ELT
Order	Transform before loading the target.	Load raw or lightly processed data, then transform in the platform.
Strength	Control what crosses the boundary.	Keep raw evidence and use scalable analytical compute.
Risk	Logic can be hidden in extraction code.	Storage, permissions, and compute costs need discipline.

The correct choice depends on governance, source constraints, latency, privacy, and the capabilities of the destination.

BhavPath-Bhavya Ponduri

DATA LAKES, WAREHOUSES, AND LAKEHOUSES

Lake	Warehouse	Lakehouse
Flexible object storage for raw and processed data in many formats. Governance and query performance require deliberate layers.	Managed analytical storage with strong SQL ergonomics, modeled tables, and controlled access.	A lake-centered architecture that adds table management, transactions, schema controls, and warehouse-like serving patterns.
Choose based on workload, users, data variety, transaction needs, governance, and operating skills rather than labels.		

BhavPath-Bhavya Ponduri

STORAGE FORMATS: LAYOUT IS PART OF PERFORMANCE

Row-oriented formats are convenient for record-at-a-time access. Column-oriented formats reduce scanned data for analytical queries that read a subset of fields.

- Use typed schemas instead of relying on implicit text parsing.
- Prefer compression and encoding that match the data types.
- Track file sizes and avoid a large number of tiny files.
- Preserve schema evolution rules and metadata alongside the data.

A format choice should support the dominant access pattern while keeping data portable enough for recovery and migration.

BhavPath-Bhavya Ponduri

PARTITIONING AND CLUSTERING: REDUCE UNNECESSARY WORK

Partitioning organizes data into independently prunable regions, commonly by a time or tenant dimension. Clustering or sorting improves locality within those regions.

Good partition key

Frequently filtered, reasonably balanced, stable, and aligned with retention or ingestion behavior.

Common failure

High-cardinality keys create tiny partitions, while a single hot key creates skew.

Validate with query plans and scan behavior; do not optimize partitioning from intuition alone.

BhavPath-Bhavya Ponduri

DATA MODELING: MAKE QUESTIONS EASY TO ASK

Modeling turns source-shaped data into structures that reflect business processes and analytical questions.

- **Grain:** state exactly what one row represents.
- **Facts:** store measurable events or transactions at a declared grain.
- **Dimensions:** provide descriptive context such as customer, product, or date.
- **Keys:** distinguish source identifiers from durable warehouse identifiers.

Good models reduce repeated business logic, expose assumptions, and make tests and ownership clear.

BhavPath-Bhavya Ponduri

SLOWLY CHANGING DIMENSIONS: PRESERVE THE STORY OF AN ENTITY

When descriptive attributes change, decide whether consumers need the current value, the historical value, or both.

PATTERN	BEHAVIOR	USE WHEN
Current-only	Overwrite the attribute.	History is not analytically relevant.
Versioned	Insert a new row with effective dates or version markers.	Reports must reproduce historical context.
Hybrid	Keep selected current and historical fields.	Consumers need both convenience and auditability.

BhavPath-Bhavya Ponduri

DISTRIBUTED PROCESSING: DIVIDE WORK WITHOUT LOSING MEANING

Distributed engines split data and computation across workers. The system must coordinate data movement, retries, failures, and the final assembly of results.

- Parallelism improves throughput when work is balanced.
- Shuffles move data between workers and can dominate runtime.
- Skew creates stragglers when one key owns disproportionate work.
- Serialization, network transfer, and small tasks add overhead.

Start with a clear physical plan: where is data read, repartitioned, joined, aggregated, and written?

BhavPath-Bhavya Ponduri

APACHE SPARK CONCEPTS TO REASON ABOUT

Driver and executors

The driver plans work; executors run tasks and hold intermediate data. Resource sizing should follow the workload and its shuffle behavior.

Lazy evaluation

Transformations build a plan. Actions trigger execution, which makes the physical plan worth inspecting.

Partitions

Partitions define units of parallel work. Too few underuse resources; too many create scheduling and I/O overhead.

Joins and shuffles

Choose join strategies with data size, key distribution, and memory constraints in mind.

BhavPath-Bhavya Ponduri

KAFKA CONCEPTS: DURABLE LOGS FOR EVENT MOVEMENT

Kafka organizes records into topics split into partitions. Producers append records; consumers track offsets and process at their own pace.

- **Partition key:** controls placement and local ordering.
- **Consumer group:** shares partitions across cooperating consumers.
- **Offset:** identifies a position and supports replay.
- **Retention:** defines how long the log remains available for recovery or new consumers.

Design for duplicate delivery, schema compatibility, backpressure, dead-letter handling, and replay safety.

BhavPath-Bhavya Ponduri

ORCHESTRATION: MAKE DEPENDENCIES AND STATE EXPLICIT

An orchestrator schedules work, tracks runs, applies dependencies, exposes retries, and helps operators understand what happened.

CONCERN	DESIGN QUESTION
Dependency	Which upstream data or condition must be ready?
Retry	Which failures are transient, and is the task idempotent?
Backfill	Can a historical interval run without disturbing current data?
Alerting	Who acts, with what context, and within what response window?

BhavPath-Bhavya Ponduri

DBT-STYLE TRANSFORMATIONS: SQL WITH SOFTWARE DISCIPLINE

A dbt-style workflow treats transformations as versioned models with dependencies, documentation, tests, and repeatable builds.

- Keep staging models close to source semantics.
- Use intermediate models to isolate reusable business rules.
- Publish marts with stable names, grains, and owner expectations.
- Use references to build a dependency graph rather than hidden table order.
- Document assumptions where a future maintainer would otherwise guess.

The value is not the tool name; it is the combination of modular SQL, lineage, tests, and review.

BhavPath-Bhavya Ponduri

DATA QUALITY: DEFINE WHAT “CORRECT” MEANS

Quality checks should reflect how the data is used. A passing pipeline can still produce harmful outputs if its assertions are too weak or disconnected from decisions.

Structural checks

Schema, types, nullability, uniqueness, referential integrity, and accepted values.

Behavioral checks

Freshness, volume, distribution, reconciliation, and business-rule expectations.

Route invalid records to quarantine when safe, fail fast when downstream trust would be compromised, and always make the response visible.

BhavPath-Bhavya Ponduri

GOVERNANCE AND SECURITY: MAKE SAFE USE THE EASY PATH

- Assign owners and stewards for important datasets.
- Classify sensitive fields and minimize collection where possible.
- Apply least-privilege access at the storage, table, row, and column levels.
- Encrypt data in transit and at rest through platform controls.
- Separate development, test, and production access and credentials.
- Record retention, deletion, legal hold, and approved-use expectations.

Governance is operational: policies need metadata, enforcement points, reviews, and evidence that controls work.

BhavPath-Bhavya Ponduri

LINEAGE: CONNECT A DECISION BACK TO ITS EVIDENCE

Lineage maps how data moves and changes from source to output. It can be table-level, column-level, operational, or business-oriented.

Why teams need it

Assess change impact, explain metric definitions, trace incidents, support audits, and find unused assets.

What to capture

Inputs, transformations, owners, run metadata, versions, freshness, and downstream consumers.

Keep lineage close to the code and execution metadata so it remains current rather than becoming a static diagram.

BhavPath-Bhavya Ponduri

OBSERVABILITY: KNOW WHEN THE SYSTEM IS DRIFTING

Observability helps operators infer system state from its outputs. Data systems need technical signals and data-aware signals together.

SIGNAL	EXAMPLES	OPERATOR QUESTION
Freshness	Last successful arrival or update time	Is the consumer seeing current data?
Volume	Rows, bytes, partitions, event counts	Did input or output behavior change?
Runtime	Duration, retries, queue time, resource use	Is the system healthy and efficient?

BhavPath-Bhavya Ponduri

TESTING DATA SYSTEMS AT MULTIPLE LAYERS

Unit tests

Validate parsing, mapping, key logic, windowing, and edge cases in isolation.

Contract tests

Verify producer and consumer assumptions about schemas and semantics.

Integration tests

Run realistic reads, writes, permissions, and orchestration dependencies.

Data tests

Assert uniqueness, relationships, ranges, freshness, reconciliation, and business rules.

Tests should fail with an actionable message and identify the affected dataset, partition, run, or code version.

BhavPath-Bhavya Ponduri

CI/CD FOR PIPELINES AND TRANSFORMATIONS

Continuous integration catches errors before deployment; continuous delivery makes a tested change repeatable to release.

- Lint SQL, configuration, schemas, and pipeline definitions.
- Run unit, contract, and representative data tests on pull requests.
- Build isolated environments or schemas for safe validation.
- Review permissions, migrations, dependencies, and rollback behavior.
- Promote immutable artifacts with versioned configuration.

Deployment is not complete until the new version is observable and the previous version can be recovered or safely superseded.

BhavPath-Bhavya Ponduri

COST OPTIMIZATION: REDUCE WASTE WITHOUT WEAKENING TRUST

Cost is a design constraint across storage, compute, network transfer, orchestration, and human response time.

Reduce avoidable work

Prune columns and partitions, reuse curated outputs, compact files, and avoid repeated full scans.

Measure the trade-off

Compare freshness, reliability, performance, and operator effort before changing a workload.

Tag workloads and datasets, review idle resources, and make cost visible to the teams that can change the design.

BhavPath-Bhavya Ponduri

FAILURE RECOVERY: DESIGN THE SECOND ATTEMPT

Failures are expected: sources time out, schemas change, workers disappear, credentials expire, and downstream systems fall behind.

- Make steps idempotent or protect them with deduplication and transactional writes.
- Checkpoint progress so retries do not restart unnecessary work.
- Keep raw inputs and manifests long enough to replay safely.
- Separate bad records from infrastructure failures.
- Define recovery objectives, escalation paths, and ownership before an incident.

Practice recovery with controlled drills; a runbook that has never been exercised is only a hypothesis.

BhavPath-Bhavya Ponduri

ARCHITECTURE CASE STUDY: RELIABLE ORDER ANALYTICS

Imagine an order platform that needs daily finance reconciliation and a near-real-time operations view.

Design

Capture order changes through a durable event path, land immutable source evidence, and maintain a warehouse model with declared order and line-item grains.

The same source can serve different decisions when each serving model states its latency, accuracy, and recovery expectations.

Controls

Use contract checks at ingestion, deduplicate by business key, reconcile totals, expose lineage, and alert on freshness or volume drift.

BhavPath-Bhavya Ponduri

HANDS-ON EXERCISES: BUILD BEFORE YOU OPTIMIZE

1. Draw a source-to-consumer map for a business question and label owners, latency, and sensitivity.
2. Define a contract for one event, including required fields, keys, timestamps, and compatibility rules.
3. Design a batch ingestion job with a cursor, manifest, retry policy, and backfill path.
4. Model a fact table and two dimensions; write the grain and three quality tests.
5. Sketch an incident runbook for a late source, a duplicate load, and a breaking schema change.

For each exercise, explain what evidence would convince a reviewer that the design is safe to operate.

BhavPath-Bhavya Ponduri

INTERVIEW QUESTIONS: EXPLAIN TRADE-OFFS, NOT TOOL SLOGANS

Design

How would you ingest changing source data? When would you choose batch over streaming? How would you make a pipeline replayable?

Performance

What causes a distributed join to slow down? How would you investigate skew, shuffles, or small files?

Trust

Which tests protect a metric? How do you handle late, duplicate, or invalid records?

Operations

What happens when a dependency fails? How do you deploy, monitor, roll back, and communicate?

Strong answers state assumptions, identify failure modes, and connect technical choices to the consumer decision.

BhavPath-Bhavya Ponduri

FINAL CHECKLIST: PRODUCTION READINESS REVIEW

- Purpose, consumers, owners, and service expectations are documented.
- Source contracts, schema evolution, and replay behavior are defined.
- Storage format, partitioning, retention, and access controls fit the workload.
- Transformations have declared grain, dependencies, tests, and documentation.
- Quality, freshness, volume, lineage, and runtime signals are visible.
- CI/CD validates code, data, permissions, and rollback paths.
- Retries are safe; failures are classified; recovery has been rehearsed.
- Costs are attributed, reviewed, and balanced against reliability.

Reliable data engineering is a continuous practice: make assumptions explicit, automate evidence, and improve the system after every failure.

BhavPath-Bhavya Ponduri